



HuggingFace Transformers: API, Models & Fine-tuning Techniques

Day 3 — Module 5 & Module 6

Module 5

The Trainer API and Full Fine-Tuning

Module 6

Training Without the Trainer — Custom PyTorch Loops

Instructor: Chandrashekar Babu <training@chandrashekar.info>

<https://www.slashprog.com/> | <https://www.chandrashekar.info/>

Day 3 Agenda

MODULE 5

The Trainer API & Full Fine-Tuning

01

Fine-Tuning Fundamentals

Transfer learning for Transformers, and the fine-tuning process overview

02

Data Preparation Pipeline

Dataset loading and exploration, tokenisation strategies, train/validation/test splits

03

The Trainer API Deep Dive

TrainingArguments configuration, metrics and evaluation, callbacks and customisation

04

Hands-on Lab

Full fine-tuning BERT for text classification on the IMDB dataset

MODULE 6

Training Without the Trainer

01

When to Use Custom Loops

Trade-offs between Trainer API and manual PyTorch control

02

Anatomy of a Training Loop

Forward pass, backward pass, optimisers, and schedulers

03

Gradient Accumulation & Mixed Precision

Techniques for memory efficiency and training speed

04

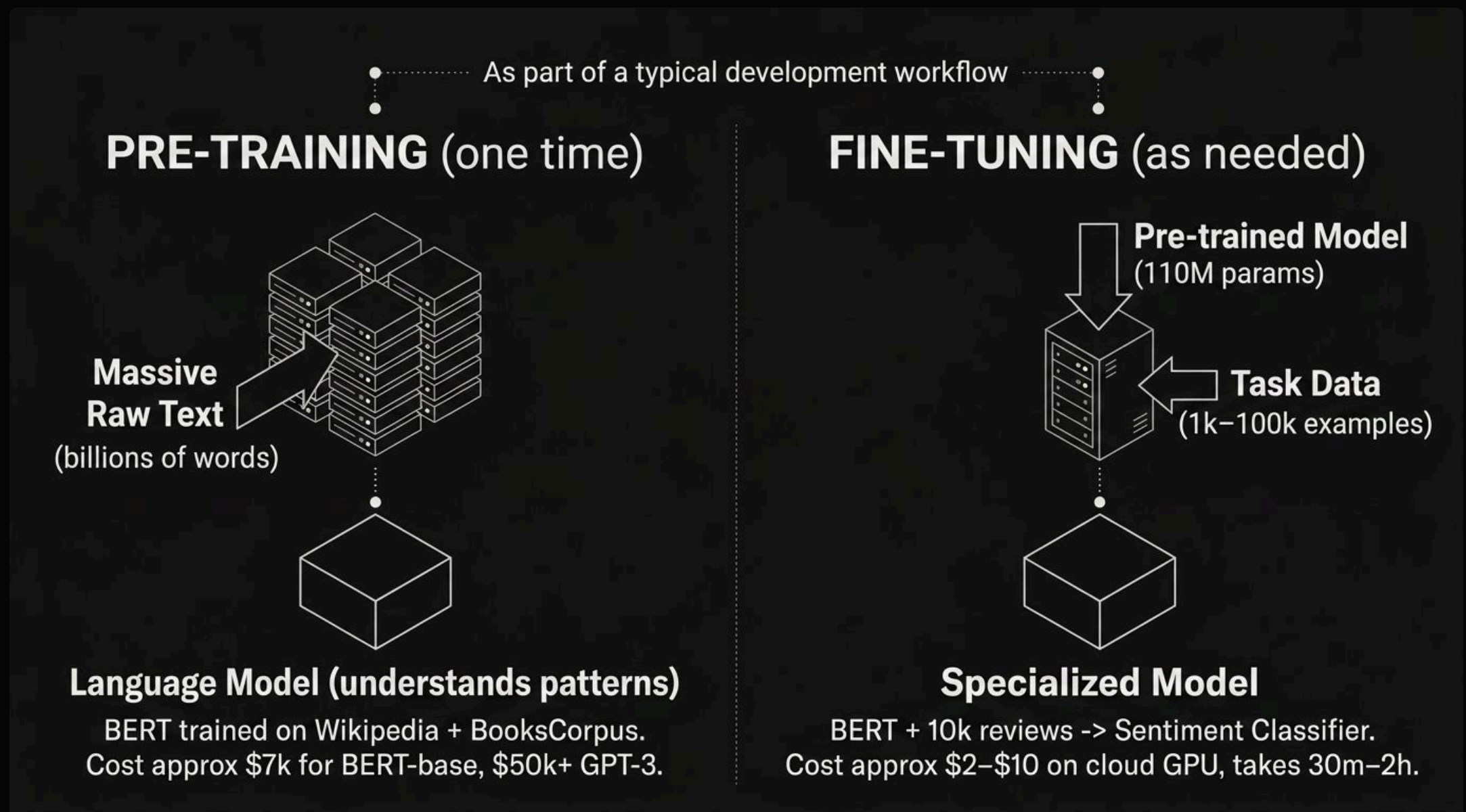
Hands-on Lab

Custom fine-tuning loop with all optimisations applied

What is Fine-Tuning?

Transfer Learning for NLP

Fine-tuning is the process of taking a large, general-purpose language model that was pre-trained on massive corpora and adapting it to a specific downstream task using a comparatively small, labelled dataset. This approach dramatically reduces the time, data, and compute required to build high-performing NLP models.

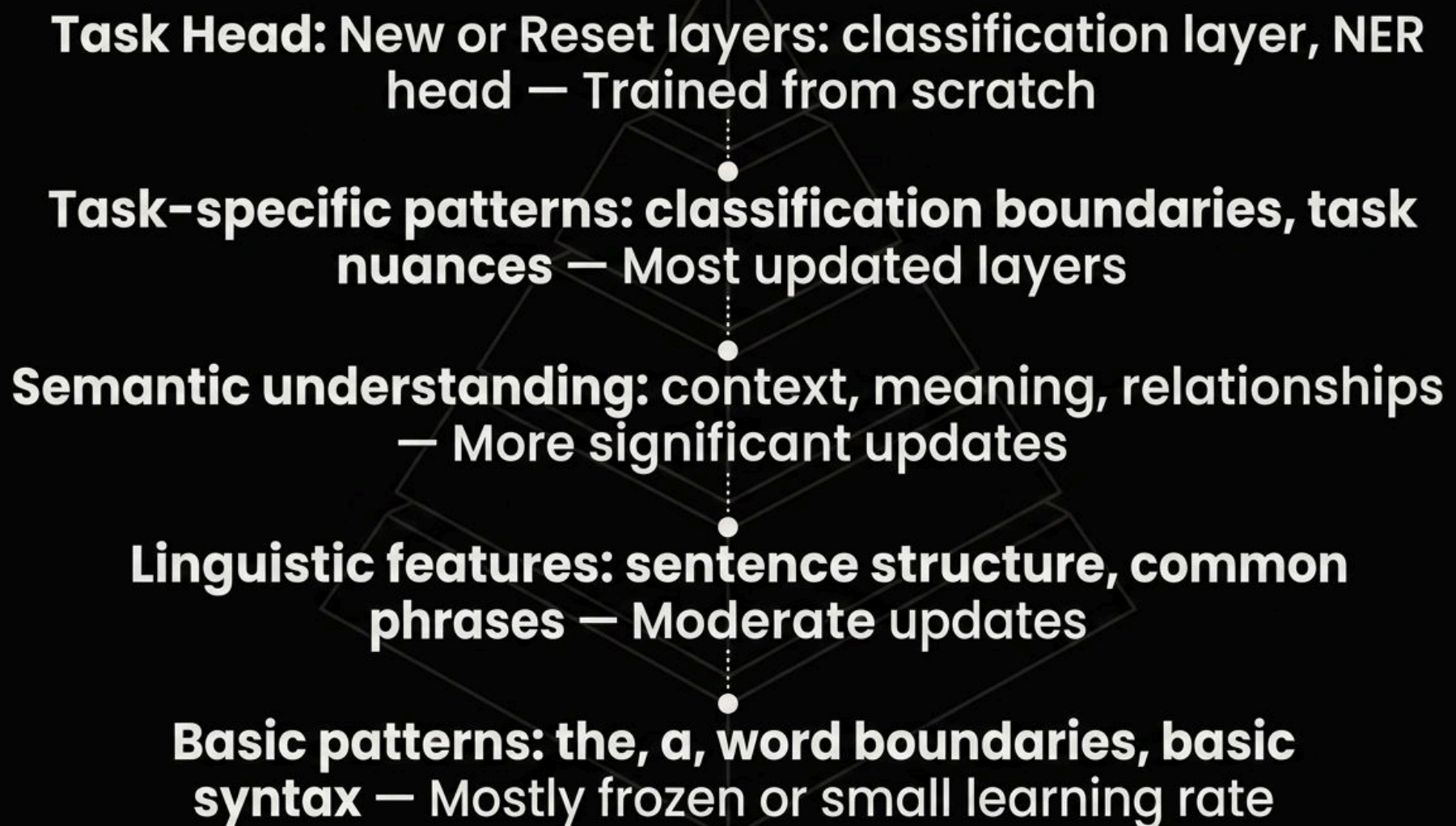


The key insight is that pre-training is extraordinarily expensive and is done once by large organisations. Fine-tuning reuses those learned representations and adapts only the top layers (plus a new task head) to your specific problem — making powerful NLP accessible to any team with a single GPU.

Why Fine-Tuning Works

The Knowledge Hierarchy in Neural Networks

Transformer models learn a hierarchy of linguistic representations distributed across their layers. Each group of layers captures a progressively more abstract level of understanding. Fine-tuning exploits this hierarchy by updating each group with a different intensity.



Analogy: Medical Training

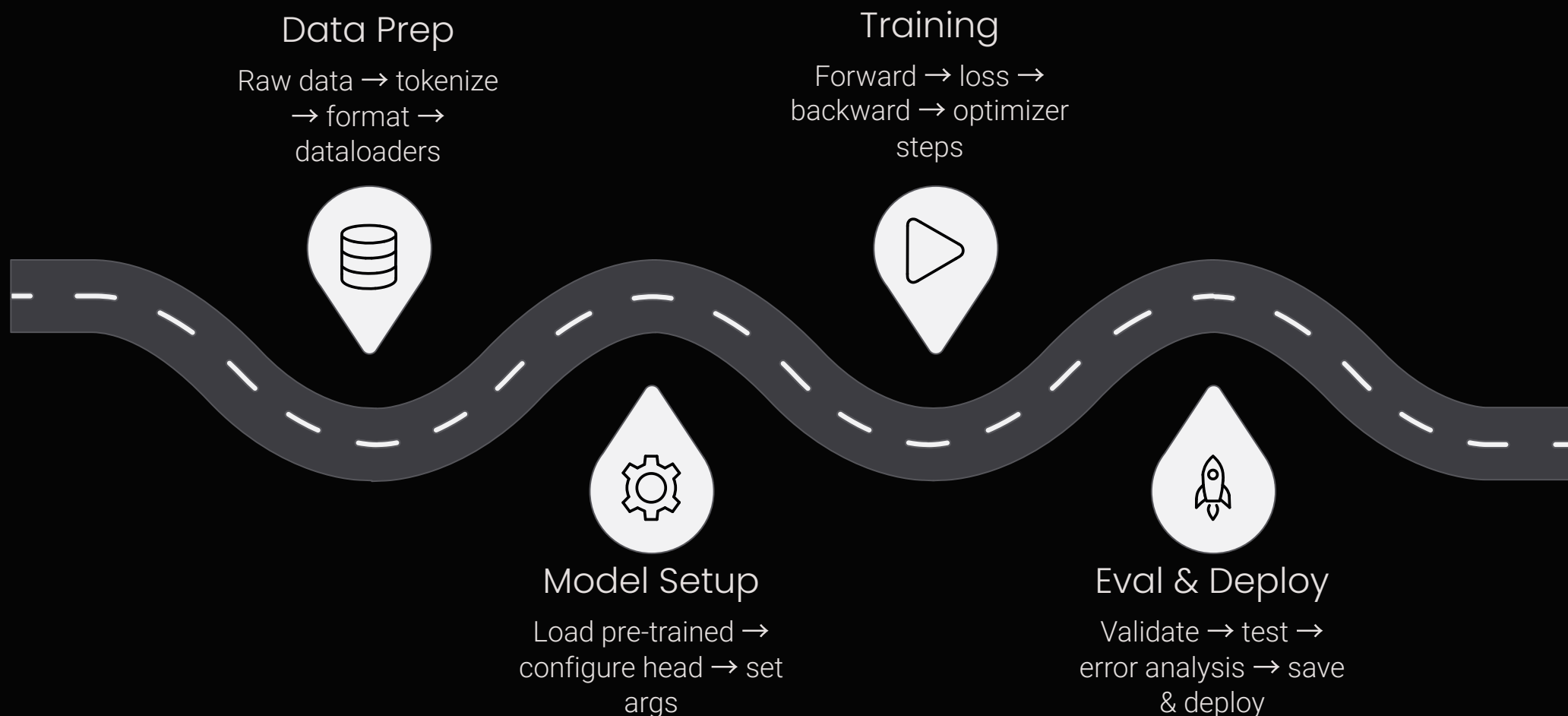
- **Pre-trained Model** = Medical student (broad, general knowledge about how the human body works)
- **Fine-Tuned Model** = Cardiologist (deep specialised expertise in a specific domain)
- **Task Head** = New certification exam (a specific test that validates the specialisation)

Practical Implication

Because lower layers capture universal language properties, they require only minimal updates during fine-tuning. The upper layers and the newly initialised task head absorb most of the gradient signal from your labelled data. This is why fine-tuning converges quickly — often in 2–5 epochs — even on datasets with only a few thousand examples.

The Fine-Tuning Pipeline

Complete Workflow — Five Stages



Each stage feeds directly into the next, forming a continuous pipeline. Data preparation ensures the model receives correctly formatted inputs. Model setup attaches the task-specific classification head to the frozen or partially frozen backbone. The training loop iteratively minimises the task loss. Evaluation tracks generalisation. Finally, saving and deploying makes the fine-tuned model available for production inference via HuggingFace Hub or a local pipeline.

Data Preparation: Loading Datasets

The `datasets` library from HuggingFace provides a unified interface for loading data from the Hub, local files, or in-memory Python objects. All loading methods return a `DatasetDict` or `Dataset` object that supports efficient batched operations, streaming, and caching.

```
from datasets import load_dataset, Dataset, DatasetDict

# --- From HuggingFace Hub ---
dataset = load_dataset("imdb")
print(dataset)
# DatasetDict({
#   train: Dataset({ features: ['text', 'label'], num_rows: 25000 })
#   test: Dataset({ features: ['text', 'label'], num_rows: 25000 })
# })

# --- From CSV / JSON ---
dataset = load_dataset("csv", data_files="my_data.csv")
dataset = load_dataset("json", data_files="my_data.json")

# --- From a pandas DataFrame ---
import pandas as pd
df = pd.read_csv("data.csv")
dataset = Dataset.from_pandas(df)

# --- Create a custom in-memory dataset ---
data = {
    "text": ["I love this!", "Terrible product.", "It was okay."],
    "label": [1, 0, 1]
}
dataset = Dataset.from_dict(data)
```

 Use `load_dataset` with `streaming=True` for very large datasets that do not fit in RAM. Streaming yields batches lazily without downloading the full corpus.

Dataset Exploration and Analysis

Before tokenising, always inspect the dataset to understand class balance, text length distribution, and potential data quality issues. Imbalanced classes or extremely long sequences will directly impact model performance and training time.

```
from collections import Counter
import numpy as np
import random

# --- Dataset structure ---
print(f"Features:      {dataset['train'].features}")
print(f"Number of samples: {len(dataset['train'])}")

# --- Label distribution ---
labels = dataset['train']['label']
print(f"Label distribution: {Counter(labels)}")

# --- Examine random samples ---
indices = random.sample(range(len(dataset['train'])), 3)
for i in indices:
    sample = dataset['train'][i]
    print(f"\nSample {i}:")
    print(f"  Text: {sample['text'][:100]}...")
    print(f"  Label: {sample['label']}")

# --- Text length statistics ---
text_lengths = [len(text.split()) for text in dataset['train']['text']]
print(f"\nText length stats:")
print(f"  Mean:      {np.mean(text_lengths):.1f} words")
print(f"  Median:    {np.median(text_lengths):.1f} words")
print(f"  95th percentile: {np.percentile(text_lengths, 95):.1f} words")
print(f"  Max:      {np.max(text_lengths)} words")
```

Pay particular attention to the 95th-percentile length — this informs your choice of `max_length` during tokenisation. Setting `max_length` too low truncates important content; too high wastes memory and slows training.

Tokenisation for Fine-Tuning

Creating a Tokenisation Function

Tokenisation converts raw strings into integer token IDs that the model understands. The `map` method applies the tokenisation function efficiently across the entire dataset in batches, with optional multiprocessing.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

def tokenize_function(examples):
    """Tokenise text data with proper truncation and padding."""
    return tokenizer(
        examples["text"],
        padding="max_length", # Pad to max_length for consistent tensor shapes
        truncation=True,      # Truncate sequences longer than max_length
        max_length=512,        # BERT's maximum sequence length
        return_tensors=None    # Return lists (will be converted later)
    )

# Apply tokenisation to the entire dataset
tokenized_datasets = dataset.map(
    tokenize_function,
    batched=True,             # Process multiple examples at once
    remove_columns=["text"],  # Remove original text (keep labels)
    desc="Tokenizing dataset" # Progress bar description
)

print(tokenized_datasets["train"][0].keys())
# dict_keys(['input_ids', 'token_type_ids', 'attention_mask', 'label'])
```

❏ Setting `batched=True` can be 10–100× faster than processing examples individually, as the tokeniser's Rust backend processes entire batches in a single call.

Tokenisation Best Practices

Handling Different Scenarios

```

from transformers import DataCollatorWithPadding
from sklearn.model_selection import train_test_split

# ——— 1. Dynamic Padding (most efficient) ———
def tokenize_dynamic(examples):
    return tokenizer(
        examples["text"],
        padding=False,  # Don't pad yet — sequences stay variable length
        truncation=True,
        max_length=512,
    )

# Pad later per-batch using DataCollator (avoids wasted padding tokens)
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

# ——— 2. Stratified Splitting ———
# For datasets that lack a built-in validation split
train_valid = dataset["train"].train_test_split(
    test_size=0.2,
    stratify_by_column="label", # Maintain class distribution across splits
    seed=42
)
# train_valid["train"] → training set
# train_valid["test"]  → validation set

# ——— 3. Handling Multi-Column Inputs (e.g., NLI, QA) ———
def preprocess_function(examples):
    return tokenizer(
        examples["premise"],
        examples["hypothesis"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

```

Dynamic Padding

Pads each *batch* to the length of its longest sequence rather than to the global maximum. This is the recommended approach — it reduces unnecessary computation on short examples and is compatible with DataCollator.

Stratified Splits

Ensures that each split contains a proportional representation of every class. Essential for imbalanced datasets where random splits could yield very different class ratios between train and validation.

Model Setup

Loading and Configuring the Model for Your Task

HuggingFace's `AutoModelForSequenceClassification` attaches a randomly initialised linear classification head on top of the pre-trained BERT encoder. Only the head is reset; all encoder weights retain their pre-trained values.

```
from transformers import AutoModelForSequenceClassification

# Load pre-trained model with a custom task head
model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2,                # Binary classification
    # problem_type="single_label_classification", # Default
    # problem_type="multi_label_classification", # Multi-label tasks
    # problem_type="regression",           # Regression tasks
    id2label={0: "NEGATIVE", 1: "POSITIVE"}, # Human-readable label map
    label2id={"NEGATIVE": 0, "POSITIVE": 1},
)

# Inspect model architecture
print(model)
# BertForSequenceClassification(
#   (bert): BertModel(...)
#   (dropout): Dropout(p=0.1, inplace=False)
#   (classifier): Linear(in_features=768, out_features=2, bias=True)
# )

# Count trainable parameters
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Total params: {total_params:,}")
print(f"Trainable: {trainable_params:,} ({100 * trainable_params / total_params:.1f}%)")
```

 For BERT-base, all 110M parameters are trainable by default during full fine-tuning. This is in contrast to parameter-efficient methods like LoRA, which freeze the backbone and only train a tiny fraction of parameters.

Understanding TrainingArguments

Key Configuration Parameters — The Complete Reference

`TrainingArguments` is a dataclass that encapsulates every aspect of training: where to save artefacts, batch sizes, learning rate schedules, evaluation cadence, and hardware optimisations. Understanding each group of parameters is essential for reproducible, efficient training.

```
from transformers import TrainingArguments
```

```
training_args = TrainingArguments(
```

```
# ——— Output and Logging ———
```

```
output_dir="./results", # Directory for model checkpoints
logging_dir="./logs", # TensorBoard logs directory
logging_steps=50, # Log metrics every N steps
report_to="tensorboard", # Also: "wandb", "mlflow", "none"
```

```
# ——— Training Duration ———
```

```
num_train_epochs=3, # Full passes through the training data
max_steps=-1, # Override epochs with step count (-1 = disabled)
```

```
# ——— Batch Sizes ———
```

```
per_device_train_batch_size=16, # Batch size per GPU during training
per_device_eval_batch_size=64, # Larger for eval (no gradients stored)
gradient_accumulation_steps=2, # Effective batch = 16 * 2 = 32
```

```
# ——— Optimisation ———
```

```
learning_rate=2e-5, # Standard starting LR for BERT fine-tuning
weight_decay=0.01, # L2 regularisation coefficient
warmup_steps=500, # Linear LR warmup steps
lr_scheduler_type="linear", # Also: "cosine", "constant", etc.
```

```
# ——— Evaluation and Checkpointing ———
```

```
evaluation_strategy="steps", # "epoch", "steps", or "no"
eval_steps=500, # Evaluate every N steps
save_strategy="steps", # Align with evaluation_strategy
save_steps=500, # Save checkpoint every N steps
save_total_limit=2, # Keep only the last 2 checkpoints
```

```
# ——— Performance ———
```

```
fp16=True, # Mixed precision (requires GPU)
dataloader_num_workers=4, # Parallel data loading workers
dataloader_pin_memory=True, # Faster CPU → GPU data transfer
```

```
# ——— Best Model Selection ———
```

```
load_best_model_at_end=True, # Restore best checkpoint after training
metric_for_best_model="accuracy", # Metric used to identify "best"
greater_is_better=True, # Higher metric value = better model
seed=42, # Reproducibility seed
)
```

TrainingArguments Quick Reference

Configurations for Different Scenarios

Quick Experimentation

Use this minimal configuration when you want to verify that your pipeline runs end-to-end before committing to a full training run. Disable evaluation and reporting to reduce overhead.

```
training_args = TrainingArguments(  
    output_dir="./quick-test",  
    num_train_epochs=1,  
    per_device_train_batch_size=8,  
    logging_steps=10,  
    evaluation_strategy="no",  
    report_to="none",  
)
```

Production Training

A robust configuration with gradient accumulation, cosine warmup, frequent evaluation, early stopping support, and dual logging backends (TensorBoard + W&B). Use this as your starting point for real tasks.

```
training_args = TrainingArguments(  
    output_dir="./production-model",  
    num_train_epochs=5,  
    per_device_train_batch_size=32,  
    gradient_accumulation_steps=4,  
    learning_rate=3e-5,  
    warmup_ratio=0.1,  
    weight_decay=0.01,  
    evaluation_strategy="steps",  
    eval_steps=100,  
    save_steps=100,  
    save_total_limit=3,  
    load_best_model_at_end=True,  
    metric_for_best_model="f1",  
    fp16=True,  
    logging_first_step=True,  
    logging_steps=50,  
    report_to=["tensorboard", "wandb"],  
)
```

Evaluation Metrics

Computing Metrics Function — Two Approaches

The `compute_metrics` function is called at each evaluation step. It receives raw logits and ground-truth labels and must return a dictionary of scalar metric values. Two common patterns are shown below.

```
import numpy as np
from evaluate import load
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
```

— **Option 1:** Using the evaluate library (recommended) —

```
accuracy_metric = load("accuracy")
f1_metric       = load("f1")
precision_metric = load("precision")
recall_metric   = load("recall")

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    return {
        "accuracy": accuracy_metric.compute(
            predictions=predictions, references=labels)["accuracy"],
        "f1":      f1_metric.compute(
            predictions=predictions, references=labels,
            average="weighted")["f1"],
        "precision": precision_metric.compute(
            predictions=predictions, references=labels,
            average="weighted")["precision"],
        "recall":   recall_metric.compute(
            predictions=predictions, references=labels,
            average="weighted")["recall"],
    }
```

— **Option 2:** Using sklearn directly —

```
def compute_metrics_sklearn(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, predictions, average="weighted"
    )
    acc = accuracy_score(labels, predictions)
    return {"accuracy": acc, "f1": f1, "precision": precision, "recall": recall}
```

 The `evaluate` library (Option 1) is preferred because it provides consistent metric implementations across frameworks and supports distributed evaluation out of the box.

The Trainer Class

Putting It All Together

The `Trainer` class is the central orchestrator that combines model, data, arguments, and metrics into a single coherent training workflow. It manages the training loop, gradient updates, evaluation cadence, checkpointing, and logging — all without requiring any boilerplate PyTorch code.

```
from transformers import Trainer

trainer = Trainer(
    model=model, # Model to train
    args=training_args, # TrainingArguments
    train_dataset=tokenized_datasets["train"], # Training data
    eval_dataset=tokenized_datasets["validation"], # Validation data
    tokenizer=tokenizer, # For padding/truncation
    data_collator=DataCollatorWithPadding(tokenizer), # Dynamic padding
    compute_metrics=compute_metrics, # Evaluation metrics
    callbacks=None, # Custom callbacks (optional)
)

# ——— Start training
trainer.train()

# ——— Evaluate on test set
test_results = trainer.evaluate(tokenized_datasets["test"])
print(test_results)
# {'eval_loss': 0.234, 'eval_accuracy': 0.942, 'eval_f1': 0.941, ...}

# ——— Make predictions
predictions = trainer.predict(tokenized_datasets["test"])
print(predictions.predictions.shape) # (N, num_labels) — logits
print(predictions.label_ids.shape) # (N,) — true labels
```

Monitoring Training Progress

What to Watch During Training

Loss Curves

Training loss should decrease monotonically. Validation loss should decrease then plateau or increase — a rising validation loss is a clear sign of overfitting.

Accuracy / F1 Progression

Should increase and then stabilise. A growing gap between train and validation metrics is a key indicator of overfitting.

Learning Rate Schedule

Should follow the configured schedule (warmup ramp followed by decay). An unexpected flat LR often indicates the scheduler is not being called correctly.

Gradient Norms

Sudden spikes in gradient norm may indicate numerical instability. Apply gradient clipping (`max_grad_norm`) if norms grow beyond ~ 1.0 .

Accessing Training Logs Programmatically

```
trainer.train()
history = trainer.state.log_history

# Extract individual metric series
train_loss = [
    log["loss"]
    for log in history if "loss" in log
]
eval_loss = [
    log["eval_loss"]
    for log in history if "eval_loss" in log
]
eval_accuracy = [
    log["eval_accuracy"]
    for log in history if "eval_accuracy" in log
]
step = [
    log["step"]
    for log in history if "loss" in log
]
eval_step = [
    log["step"]
    for log in history if "eval_loss" in log
]
```

Visualising Training with TensorBoard

Launching TensorBoard and Custom Logging Callbacks

TensorBoard provides an interactive web UI for tracking loss curves, learning rate schedules, and custom metrics in real time. Combine it with a custom `TrainerCallback` to add console logging or trigger external integrations.

```
# — Launch TensorBoard
```

```
# In terminal:
```

```
# tensorboard --logdir ./logs
```

```
#
```

```
# Or from within a Jupyter notebook:
```

```
%load_ext tensorboard
```

```
%tensorboard --logdir ./logs
```

```
from transformers import TrainerCallback
```

```
class CustomLoggingCallback(TrainerCallback):
```

```
    """Prints a formatted log line to stdout at every logging step."""
```

```
    def on_log(self, args, state, control, logs=None, **kwargs):
```

```
        if logs:
```

```
            print(f"Step {state.global_step}:")
```

```
            for key, value in logs.items():
```

```
                if isinstance(value, float):
```

```
                    print(f" {key}: {value:.4f}")
```

```
# Register the callback with the Trainer
```

```
trainer.add_callback(CustomLoggingCallback())
```

 You can register multiple callbacks simultaneously. HuggingFace includes built-in callbacks for TensorBoard (`TensorBoardCallback`), W&B (`WandbCallback`), and early stopping (`EarlyStoppingCallback`).

Trainer Callbacks

Customising Training Behaviour with Three Callback Patterns

```
from transformers import TrainerCallback, EarlyStoppingCallback
```

```
# — 1. Early Stopping
```

```
# Stops training when the monitored metric fails to improve for N evaluations.
```

```
# Requires load_best_model_at_end=True in TrainingArguments.
```

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=train_dataset,  
    eval_dataset=eval_dataset,  
    compute_metrics=compute_metrics,  
    callbacks=[EarlyStoppingCallback(  
        early_stopping_patience=3, # Stop after 3 evals without improvement  
        early_stopping_threshold=0.001 # Minimum delta to qualify as improvement  
    )],  
)
```

```
# — 2. Custom Best-Model Tracker
```

```
class SaveBestModelCallback(TrainerCallback):
```

```
    def __init__(self):
```

```
        self.best_metric = 0.0
```

```
    def on_evaluate(self, args, state, control, metrics=None, **kwargs):
```

```
        if metrics and "eval_accuracy" in metrics:
```

```
            if metrics["eval_accuracy"] > self.best_metric:
```

```
                self.best_metric = metrics["eval_accuracy"]
```

```
            print(f"
```

Saving and Loading Models

Local Storage, Inference Pipelines, and HuggingFace Hub

Once training is complete, saving the model and tokeniser together ensures that anyone loading the checkpoint has all the information needed for inference. Pushing to the Hub makes the model publicly (or privately) accessible and enables easy version control.

```
from transformers import pipeline
from huggingface_hub import notebook_login

# — Save locally —————

trainer.save_model("./my-fine-tuned-model")
tokenizer.save_pretrained("./my-fine-tuned-model")

# — Load for inference using Pipeline API —————
classifier = pipeline(
    "text-classification",
    model="./my-fine-tuned-model",
    tokenizer="./my-fine-tuned-model"
)

# — Push to HuggingFace Hub —————

notebook_login() # Authenticate with your HF token

trainer.push_to_hub(
    commit_message="Fine-tuned BERT on custom sentiment dataset",
    tags=["sentiment-analysis", "bert", "custom-data"],
)

# — Generate a model card automatically —————
trainer.create_model_card(
    language="en",
    license="mit",
    tags=["sentiment-analysis"],
    model_description="Fine-tuned BERT for custom sentiment classification",
)
```

- ❏ Always push both the model and the tokeniser. A mismatched tokeniser (e.g., different vocabulary or special tokens) will produce incorrect inputs at inference time, silently degrading model performance.

Common Fine-Tuning Issues & Solutions

Issue 1: Out of Memory (OOM)

```
training_args = TrainingArguments(
    per_device_train_batch_size=8, # Reduce: 16 → 8 → 4
    gradient_accumulation_steps=4, # Compensate with
    accumulation
    fp16=True, # Enable mixed precision
    gradient_checkpointing=True, # Trade compute for
    memory
    optim="adamw_8bit", # 8-bit optimizer
    (bitsandbytes)
)
model.gradient_checkpointing_enable()
```

Issue 2: Overfitting

```
training_args = TrainingArguments(
    weight_decay=0.1, # Increase L2 regularisation
    num_train_epochs=2, # Reduce training duration
    warmup_ratio=0.1, # Add warmup phase
    learning_rate=1e-5, # Lower learning rate
    load_best_model_at_end=True, # Use best checkpoint
)
model.config.hidden_dropout_prob = 0.2
model.config.attention_probs_dropout_prob = 0.2
```

Issue 3: Slow Training

```
training_args = TrainingArguments(
    fp16=True, # Mixed precision (~2× speedup)
    dataloader_num_workers=8, # Parallel data loading
    dataloader_pin_memory=True, # Pinned memory for
    GPU transfer
    per_device_train_batch_size=32, # Larger batches = higher
    throughput
    tf32=True, # TF32 on Ampere GPUs
)
# Pre-tokenise once, reuse across runs:
tokenized_datasets.save_to_disk("./tokenized_data")
```

Issue 4: Unstable Training (NaN Loss)

```
training_args = TrainingArguments(
    learning_rate=1e-5, # Lower LR stabilises training
    warmup_steps=1000, # Longer warmup period
    max_grad_norm=0.5, # Aggressive gradient clipping
    adam_beta1=0.9,
    adam_beta2=0.999,
    adam_epsilon=1e-8,
    label_smoothing_factor=0.1, # Soften hard targets
)
```

Hands-on Lab: Full Fine-Tuning Pipeline

Task: Fine-tune BERT for Sentiment Analysis on IMDB

This 60-minute lab walks through the entire fine-tuning workflow from data loading to model persistence. By the end, you will have a production-ready sentiment classifier with documented performance metrics.

Lab Goals

- Load and preprocess the IMDB dataset
- Configure BERT for sequence classification
- Set up Trainer with appropriate hyperparameters
- Train the model and monitor progress
- Evaluate on the test set
- Save and test the fine-tuned model

Lab Exercises

1. Data Loading and Exploration
2. Tokenisation
3. Model Setup
4. Training Configuration and Execution
5. Evaluation and Analysis
6. Error Analysis
7. Save and Test the Model

Expected Outcome

A fine-tuned `bert-base-uncased` model achieving approximately **92–94% accuracy** on the IMDB test set, trained on a 5,000-sample subset in under 30 minutes on a single GPU.

Lab Exercise 1: Data Loading and Exploration

```
from datasets import load_dataset
from transformers import AutoTokenizer
import numpy as np
from collections import Counter

print("=" * 60)
print("EXERCISE 1: Data Loading and Exploration")
print("=" * 60)

# ——— 1. Load the full IMDB dataset ———
dataset = load_dataset("imdb")
print(f"\nDataset structure: {dataset}")
print(f"Train size: {len(dataset['train'])}")
print(f"Test size: {len(dataset['test'])}")

# ——— 2. Explore a sample ———
print(f"\nFeatures: {dataset['train'].features}")
print(f"Sample 0 text: {dataset['train'][0]['text'][:200]}...")
print(f"Sample 0 label: {dataset['train'][0]['label']}")

# ——— 3. Check class distribution ———
train_labels = dataset['train']['label']
print(f"\nTraining set class distribution: {Counter(train_labels)}")

# ——— 4. Create a small subset for faster training ———
small_train = dataset['train'].select(range(5000))
small_test = dataset['test'].select(range(1000))

# Split train into train / validation (80 / 20)
split_dataset = small_train.train_test_split(test_size=0.2, seed=42)
final_dataset = {
    'train': split_dataset['train'],
    'validation': split_dataset['test'],
    'test': small_test,
}

print(f"\nFinal dataset sizes:")
print(f" Train: {len(final_dataset['train'])}")
print(f" Validation: {len(final_dataset['validation'])}")
print(f" Test: {len(final_dataset['test'])}")
```

Lab Exercise 2: Tokenisation

```
print("\n" + "=" * 60)
print("EXERCISE 2: Tokenization")
print("=" * 60)

# ——— 1. Load tokeniser ———
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# ——— 2. Define tokenisation function ———
def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=256, # 256 instead of 512 for faster training
    )

# ——— 3. Tokenise all splits ———
tokenized_datasets = {}
for split, data in final_dataset.items():
    print(f"\nTokenizing {split} split...")
    tokenized_datasets[split] = data.map(
        tokenize_function,
        batched=True,
        remove_columns=["text"],
        desc=f"Tokenizing {split}"
    )

# ——— 4. Inspect a tokenised sample ———
sample = tokenized_datasets['train'][0]
print(f"\nTokenized sample keys: {sample.keys()}")
print(f"Input IDs (first 20): {sample['input_ids'][:20]}")
print(f"Input IDs length: {len(sample['input_ids'])}")
print(f"Attention mask sum: {sum(sample['attention_mask'])}")
print(f"Label: {sample['label']}")

# ——— 5. Decode to verify round-trip ———
decoded = tokenizer.decode(sample['input_ids'], skip_special_tokens=True)
print(f"\nDecoded text (first 100 chars): {decoded[:100]}...")
```

Lab Exercise 3: Model Setup

```
print("\n" + "=" * 60)
print("EXERCISE 3: Model Setup")
print("=" * 60)

from transformers import AutoModelForSequenceClassification
import torch

# ——— 1. Load BERT with classification head ———
model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2,
    id2label={0: "NEGATIVE", 1: "POSITIVE"},
    label2id={"NEGATIVE": 0, "POSITIVE": 1},
)

# ——— 2. Parameter analysis ———

total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"\nTotal parameters: {total_params:,}")
print(f"Trainable parameters: {trainable_params:,}")
print(f"Percentage trainable: {100 * trainable_params / total_params:.1f}%")

# ——— 3. Move to available device ———
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"\nModel on device: {next(model.parameters()).device}")

# ——— 4. Quick sanity-check forward pass ———
test_input = tokenizer(
    "This movie was fantastic!",
    return_tensors="pt",
    padding=True,
    truncation=True,
    max_length=256
).to(device)

with torch.no_grad():
    outputs = model(**test_input)
    logits = outputs.logits
    predictions = torch.softmax(logits, dim=-1)

print(f"\nTest prediction:")
print(f" Logits: {logits}")
print(f" Probabilities: {predictions}")
print(f" Predicted class: {torch.argmax(predictions, dim=-1).item()}")
```

Lab Exercise 4: Training Configuration and Execution

```
print("\n" + "=" * 60)
print("EXERCISE 4: Training Configuration")
print("=" * 60)
```

```
from transformers import TrainingArguments, Trainer
from evaluate import load
import numpy as np
```

```
# ——— 1. Set up evaluation metrics ———
```

```
accuracy_metric = load("accuracy")
f1_metric = load("f1")
```

```
def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    accuracy = accuracy_metric.compute(
        predictions=predictions, references=labels)["accuracy"]
    f1 = f1_metric.compute(
        predictions=predictions, references=labels,
        average="weighted")["f1"]
    return {"accuracy": accuracy, "f1": f1}
```

```
# ——— 2. Configure training arguments ———
```

```
training_args = TrainingArguments(
    output_dir="./imdb-bert-finetuned",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    learning_rate=2e-5,
    weight_decay=0.01,
    warmup_steps=100,
    logging_dir="./logs",
    logging_steps=50,
    evaluation_strategy="steps",
    eval_steps=100,
    save_strategy="steps",
    save_steps=100,
    save_total_limit=2,
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
    fp16=torch.cuda.is_available(),
    report_to="none",
    seed=42,
)
```

```
# ——— 3. Instantiate Trainer ———
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
)
```

```
# ——— 4. Train ———
```

```
print("\nStarting training...")
trainer.train()
print("\nTraining complete!")
```

Lab Exercise 5: Evaluation and Analysis

```
print("\n" + "=" * 60)
print("EXERCISE 5: Evaluation and Analysis")
print("=" * 60)

from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# ——— 1. Evaluate on held-out test set ———
print("\nEvaluating on test set...")
test_results = trainer.evaluate(tokenized_datasets["test"])
print("\nTest Results:")
for key, value in test_results.items():
    print(f" {key}: {value:.4f}")
```

```
# ——— 2. Retrieve full prediction outputs ———
predictions_output = trainer.predict(tokenized_datasets["test"])
predictions = np.argmax(predictions_output.predictions, axis=1)
labels = predictions_output.label_ids
```

```
# ——— 3. Confusion matrix
```

```
cm = confusion_matrix(labels, predictions)
print(f"\nConfusion Matrix:\n{cm}")
```

```
plt.figure(figsize=(8, 6))
sns.heatmap(
    cm, annot=True, fmt='d', cmap='Blues',
    xticklabels=['NEGATIVE', 'POSITIVE'],
    yticklabels=['NEGATIVE', 'POSITIVE']
)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```

```
# ——— 4. Full classification report ———
print(f"\nClassification Report:")
print(classification_report(
    labels, predictions,
    target_names=['NEGATIVE', 'POSITIVE']
))
```

Lab Exercise 6: Error Analysis

```

print("\n" + "=" * 60)
print("EXERCISE 6: Error Analysis")
print("=" * 60)

# ——— 1. Identify misclassified examples ———
misclassified = [
    i for i, (pred, true) in enumerate(zip(predictions, labels))
    if pred != true
]
print(f"\nMisclassified examples: {len(misclassified)}")
print(f"Error rate:      {len(misclassified) / len(labels) * 100:.1f}%")

# ——— 2. Inspect individual errors ———
print("\nSample misclassified examples:")
for idx in misclassified[:5]:
    text    = dataset['test'][idx]['text']
    true_label = "POSITIVE" if labels[idx] == 1 else "NEGATIVE"
    pred_label = "POSITIVE" if predictions[idx] == 1 else "NEGATIVE"
    probs     = torch.softmax(
        torch.tensor(predictions_output.predictions[idx]), dim=0)
    confidence = probs[predictions[idx]].item()

    print(f"\n Example {idx}:")
    print(f"  Text:    {text[:200]}...")
    print(f"  True:    {true_label} | Predicted: {pred_label}")
    print(f"  Confidence: {confidence:.3f}")

# ——— 3. Stratify accuracy by text length ———
lengths    = [len(text.split()) for text in dataset['test']['text']]
short_texts = [i for i, l in enumerate(lengths) if l < 100]
long_texts  = [i for i, l in enumerate(lengths) if l > 500]

short_accuracy = (sum(1 for i in short_texts if labels[i] == predictions[i])
                  / len(short_texts))
long_accuracy  = (sum(1 for i in long_texts  if labels[i] == predictions[i])
                  / len(long_texts))

print(f"\nAccuracy by text length:")
print(f" Short texts (<100 words): {short_accuracy:.3f}")
print(f" Long texts  (>500 words): {long_accuracy:.3f}")

```

Analysing errors by length reveals whether the model struggles with very short reviews (insufficient signal) or very long reviews (important content truncated at 256 tokens). This informs decisions about increasing `max_length` or adding data augmentation.

Lab Exercise 7: Save and Test the Model

```

print("\n" + "=" * 60)
print("EXERCISE 7: Save and Test Model")
print("=" * 60)

from transformers import pipeline

# ——— 1. Save model and tokeniser locally ———
save_path = "./my-sentiment-model"
trainer.save_model(save_path)
tokenizer.save_pretrained(save_path)
print(f"\nModel saved to: {save_path}")

# ——— 2. Load via the Pipeline API ———
classifier = pipeline(
    "text-classification",
    model=save_path,
    tokenizer=save_path,
    device=0 if torch.cuda.is_available() else -1
)

# ——— 3. Test on custom examples ———
test_examples = [
    "This movie was absolutely fantastic! I loved every minute of it.",
    "Terrible waste of time. The acting was wooden and the plot made no sense.",
    "It was okay. Nothing special, but not terrible either.",
    "One of the best films I've ever seen. Brilliant cinematography!",
    "I fell asleep halfway through. So boring and predictable.",
    "The special effects were amazing but the story was weak.",
    "A masterpiece of modern cinema. Will watch again!",
    "Don't waste your money on this garbage.",
]

print("\nTesting on custom examples:")
for text in test_examples:
    result = classifier(text)[0]
    label = result['label']
    score = result['score']
    bar = "█" * int(score * 20)
    print(f"\n Text: {text[:80]}...")
    print(f" {label} ({score:.3f}) {bar}")

# ——— 4. Final summary ———

print("\n" + "=" * 60)
print("Summary:")
print(f" Model: bert-base-uncased fine-tuned on IMDB")
print(f" Training samples: {len(final_dataset['train'])}")
print(f" Test accuracy: {test_results['eval_accuracy']:.3f}")
print(f" Test F1: {test_results['eval_f1']:.3f}")
print(f" Model saved to: {save_path}")

```

Key Takeaways — Module 5

Fine-Tuning

Adapts pre-trained models to specific tasks with relatively little data and compute — typically hours on a single GPU versus weeks for pre-training.

Data Preparation

Involves tokenisation with appropriate padding strategies, stratified splitting to preserve class balance, and pre-tokenising to disk for efficient reuse.

TrainingArguments

Controls every aspect of training: batch size, learning rate and schedule, evaluation cadence, checkpointing, and hardware optimisations.

Trainer API

Handles the training loop, gradient updates, evaluation, logging, and checkpointing automatically — no boilerplate PyTorch required.

Metrics & Error Analysis

Use the `evaluate` library for standard metrics. Always analyse errors by class, text length, and confidence to identify model blind spots.

Common Issues

OOM, overfitting, slow training, and instability all have well-established solutions: gradient accumulation, dropout, fp16, gradient clipping, and warmup.

Additional Resources — Module 5

Official Documentation

- [**Trainer API Reference**](#) — Full parameter documentation for the Trainer class
- [**TrainingArguments Reference**](#) — All configuration options with defaults and descriptions
- [**Datasets Library**](#) — Loading, processing, and caching datasets at scale

Tutorials & Tools

- [**HuggingFace NLP Course — Chapter 3: Fine-tuning**](#) — Step-by-step guide with exercises
- [**Fine-tune a Pretrained Model**](#) — Official fine-tuning tutorial with multiple task types
- [**Weights & Biases**](#) — Experiment tracking, hyperparameter sweeps, and model registry
- [**TensorBoard**](#) — Real-time training visualisation and metric comparison

Module 6: Training Without the Trainer

Custom PyTorch training loops give you fine-grained control over every step of the training process. This module covers when to go custom, how to build a robust loop from first principles, and how to incorporate advanced techniques like gradient accumulation and mixed precision.

When to Go Custom

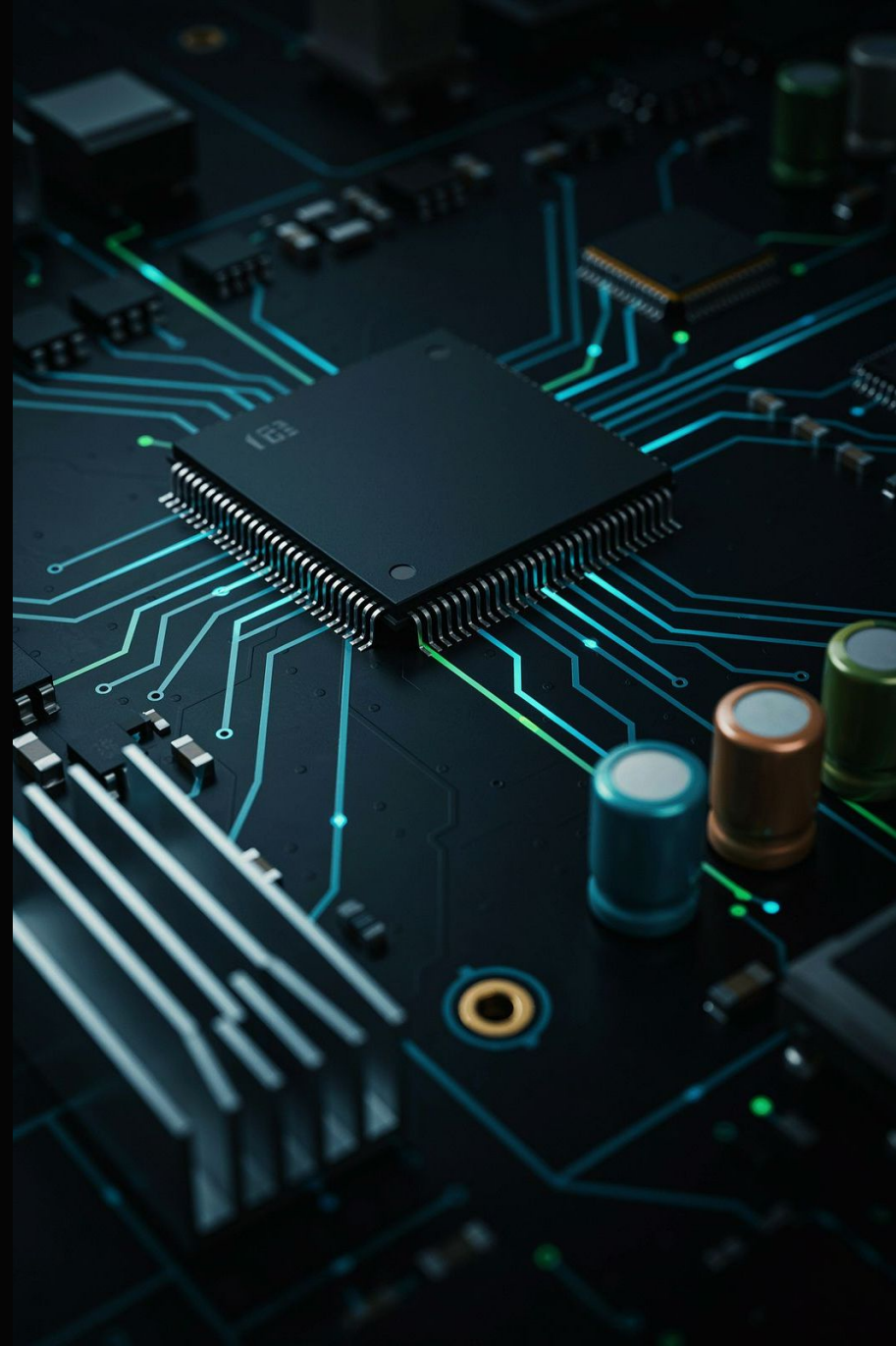
Complex loss functions, multi-task objectives, novel architectures, and research experiments that push beyond Trainer's built-in functionality.

Anatomy of the Loop

Forward pass → loss computation → backward pass → gradient clipping → optimiser step → scheduler update. Understanding each step enables precise debugging.

Optimisations

Gradient accumulation simulates large batches; mixed precision (FP16) delivers 2–3× speedup; gradient clipping prevents instability.



Why Go Custom?

Trainer API vs. Custom Training Loop — When to Use Each

✓ Use Trainer When...

Standard Supervised Learning

Classification, regression, NER, question answering, or any task well-supported by HuggingFace's Auto classes and built-in loss functions.

Rapid Prototyping

You need results quickly and can accept the Trainer's default assumptions about training dynamics and evaluation cadence.

Built-in Features Matter

You want logging, checkpointing, distributed training, and evaluation without writing any boilerplate — all included out of the box.

🔧 Go Custom When...

Complex Loss Functions

Contrastive learning, triplet loss, multi-task objectives, or any scenario where the loss is not a simple cross-entropy or MSE call.

Custom Optimisation Logic

GANs, meta-learning (MAML), reward-based fine-tuning (RLHF), or alternating optimisation schemes between model components.

Non-Standard Architectures

Models that do not fit neatly into HuggingFace's input/output conventions, or that combine HuggingFace models with external PyTorch modules.

Custom Gradient Manipulation

Gradient surgery for multi-task learning, selective gradient masking, or implementing novel regularisation techniques at the gradient level.

Anatomy of a Training Loop

The Five Core Components of Every PyTorch Loop

Every PyTorch training loop — regardless of complexity — consists of the same five operations executed in order. Understanding each step precisely is essential for debugging and extending the loop.

```
# Essential PyTorch training loop structure
```

```
# — 1. Setup
```

```
model.train()      # Switch BatchNorm and Dropout to training mode
optimizer.zero_grad() # Clear gradients from the previous step
```

```
# — 2. Forward Pass
```

```
outputs = model(input_ids, attention_mask=attention_mask)
loss     = loss_fn(outputs.logits, labels)
```

```
# — 3. Backward Pass
```

```
loss.backward()    # Compute gradients for all parameters via autograd
```

```
# — 4. Optimizer Step
```

```
optimizer.step()   # Update parameters using computed gradients
scheduler.step()   # Advance the learning rate schedule
```

```
# — 5. (Optional) Gradient Clipping
```

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
# Note: clipping must happen BEFORE optimizer.step() but AFTER loss.backward()
```

⚠ Always call `optimizer.zero_grad()` at the start of each step (or end of previous step). Forgetting to clear gradients causes them to accumulate across iterations, producing incorrect updates and often NaN losses.

Complete Training Loop Template

Train and Evaluate Functions — Reusable Patterns

```
import torch
from torch.utils.data import DataLoader
from tqdm.auto import tqdm

def train_epoch(model, dataloader, optimizer, scheduler, device):
    """Run one full pass through the training data."""
    model.train()
    total_loss = 0
    progress_bar = tqdm(dataloader, desc="Training")

    for batch in progress_bar:
        # Move entire batch to the target device
        batch = {k: v.to(device) for k, v in batch.items()}

        # Forward pass — HuggingFace models return the loss when labels are provided
        outputs = model(**batch)
        loss = outputs.loss
        total_loss += loss.item()

        # Backward pass
        loss.backward()

        # Gradient clipping (optional but recommended)
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

        # Parameter update and LR schedule step
        optimizer.step()
        scheduler.step()
        optimizer.zero_grad()

    progress_bar.set_postfix({'loss': loss.item()})

    return total_loss / len(dataloader)

def evaluate(model, dataloader, device):
    """Evaluate the model and return loss, predictions, and labels."""
    model.eval()
    total_loss = 0
    all_predictions = []
    all_labels = []

    with torch.no_grad():
        for batch in tqdm(dataloader, desc="Evaluating"):
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            total_loss += outputs.loss.item()
            predictions = torch.argmax(outputs.logits, dim=-1)
            all_predictions.extend(predictions.cpu().numpy())
            all_labels.extend(batch['labels'].cpu().numpy())

    avg_loss = total_loss / len(dataloader)
    return avg_loss, all_predictions, all_labels
```

Optimisers and Schedulers

Choosing and Configuring the Right Combination

The optimiser updates model parameters based on computed gradients, while the scheduler adjusts the learning rate over time. Choosing the right pair is as important as choosing the right architecture.

```
from transformers import get_scheduler, AdamW
from torch.optim import AdamW as TorchAdamW

# ——— Option 1: HuggingFace AdamW (recommended for Transformers) ———
# Applies weight decay correctly — excludes bias and LayerNorm parameters
optimizer = AdamW(
    model.parameters(),
    lr=2e-5,
    betas=(0.9, 0.999),
    eps=1e-8,
    weight_decay=0.01,
)

# ——— Option 2: PyTorch built-in AdamW ———
optimizer = TorchAdamW(model.parameters(), lr=2e-5, weight_decay=0.01)

# ——— Learning Rate Schedulers ———
num_training_steps = len(train_dataloader) * num_epochs
num_warmup_steps = int(0.1 * num_training_steps) # 10% warmup

# Linear schedule with warmup (most common for fine-tuning)
scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=num_warmup_steps,
    num_training_steps=num_training_steps,
)

# ——— Alternative Scheduler Types ———
# "cosine" — Cosine annealing to zero
# "cosine_with_restarts" — Cosine with periodic warm restarts
# "polynomial" — Polynomial decay
# "constant" — Fixed learning rate (no schedule)
# "constant_with_warmup" — Fixed LR after linear warmup
```

 The HuggingFace **AdamW** correctly excludes bias and LayerNorm parameters from weight decay, which is the canonical implementation for Transformer fine-tuning. Always prefer it over a naive application of weight decay to all parameters.

Gradient Accumulation

Training with Larger Effective Batch Sizes on Limited Hardware

Gradient accumulation is the standard technique for working around GPU memory constraints. Instead of updating parameters after every forward pass, gradients are accumulated over multiple steps and a single update is made at the end. The result is mathematically equivalent to training with a larger batch.

```
# Problem: Your GPU can only fit batch_size=4
# Solution: Accumulate gradients over multiple steps before updating
#
# Without accumulation: effective_batch_size = 4
# With accumulation: effective_batch_size = 4 × 8 = 32

accumulation_steps = 8 # Accumulate over 8 forward passes

for step, batch in enumerate(dataloader):
    batch = {k: v.to(device) for k, v in batch.items()}

    # Forward pass
    outputs = model(**batch)
    loss = outputs.loss / accumulation_steps #
```

Mixed Precision Training

Faster Training with FP16 — Autocast and GradScaler

Mixed precision training stores weights in 32-bit floats but performs the forward and backward pass computations in 16-bit floats. This halves memory usage and typically provides a 2–3× training speedup on modern GPUs (Volta architecture and newer), with negligible accuracy loss.

```
from torch.cuda.amp import autocast, GradScaler

# Initialise the gradient scaler (prevents underflow in FP16 gradients)
scaler = GradScaler()

for batch in dataloader:
    batch = {k: v.to(device) for k, v in batch.items()}

    # ——— Forward pass inside autocast context ———
    with autocast():
        outputs = model(**batch)
        loss = outputs.loss

    # ——— Scaled backward pass ———
    # GradScaler multiplies loss by a large scale factor before backward()
    # to prevent FP16 gradient underflow, then rescales before optimizer.step()
    scaler.scale(loss).backward()

    # ——— Unscale gradients before clipping ———
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

    # ——— Optimizer step with dynamic scale adjustment ———
    scaler.step(optimizer) # Skips update if gradients contain Inf/NaN
    scaler.update()       # Adjusts scale factor for next iteration
    scheduler.step()
    optimizer.zero_grad()
```

✓ 2–3× Faster

FP16 tensor cores on modern GPUs perform matrix multiplications much faster than FP32.

✓ 50% Less Memory

FP16 activations consume half the memory of FP32, allowing larger batches or longer sequences.

✓ Negligible Accuracy Loss

GradScaler prevents underflow; master weights remain in FP32 for numerical stability.

Full Training Script with All Optimisations

Complete Production-Ready Custom Loop

```
import torch
from transformers import AutoModelForSequenceClassification, AutoTokenizer, get_scheduler
from torch.utils.data import DataLoader
from torch.cuda.amp import autocast, GradScaler
from datasets import load_dataset
from tqdm.auto import tqdm
```

——— Hyperparameters

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model_name = "bert-base-uncased"
num_epochs = 3
batch_size = 8
accumulation_steps = 4 # Effective batch = 8 × 4 = 32
learning_rate = 2e-5
```

——— Load model and tokeniser

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=2)
model.to(device)
```

——— Prepare data

```
def tokenize(examples):
    return tokenizer(examples["text"], padding="max_length",
truncation=True, max_length=256)
```

```
dataset = load_dataset("imdb")
small_dataset = dataset["train"].select(range(2000))
tokenized = small_dataset.map(tokenize, batched=True, remove_columns=["text"])
tokenized.set_format("torch", columns=["input_ids", "attention_mask", "label"])
dataloader = DataLoader(tokenized, batch_size=batch_size, shuffle=True)
```

——— Optimiser and scheduler

```
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)
total_steps = len(dataloader) * num_epochs // accumulation_steps
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=total_steps)
scaler = GradScaler()
```

——— Training loop

```
model.train()
for epoch in range(num_epochs):
    print(f"\nEpoch {epoch + 1}/{num_epochs}")
    progress_bar = tqdm(dataloader, desc="Training")
    epoch_loss = 0
```

```
    for step, batch in enumerate(progress_bar):
        batch = {k: v.to(device) for k, v in batch.items()}
```

```
        with autocast():
            outputs = model(**batch)
            loss = outputs.loss / accumulation_steps
```

```
        scaler.scale(loss).backward()
        epoch_loss += loss.item()
```

```
    if (step + 1) % accumulation_steps == 0:
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(optimizer)
        scaler.update()
        scheduler.step()
        optimizer.zero_grad()
```

```
    progress_bar.set_postfix({'loss': loss.item() * accumulation_steps})
```

```
    avg_loss = epoch_loss / (len(dataloader) // accumulation_steps)
    print(f"Epoch {epoch + 1} average loss: {avg_loss:.4f}")
```

Debugging Training Issues

Common Problems and Targeted Solutions

Problem 1: Loss is NaN

```
# Causes: LR too high, gradient explosion, or NaN values in input data
# Solutions:
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=0.5)
# Enable anomaly detection to pinpoint the operation that produced NaN
torch.autograd.set_detect_anomaly(True)
# Inspect inputs for NaN before each forward pass
assert not torch.isnan(batch['input_ids'].float()).any()
```

Problem 2: Loss Not Decreasing

```
# Check these common mistakes:
print(f"Model training mode: {model.training}") # Must be True
print(f"Sample labels: {batch['labels'][:5]}")
print(f"Sample predictions: {outputs.logits.argmax(-1)[:5]}")
# LR may be too small — try 1e-4 briefly to confirm gradients flow
# Verify labels are 0-indexed integers matching num_labels
```

Problem 3: GPU Memory Leak

```
# Cause: Accumulating tensors (e.g., appending loss to a list)
# Wrong:
losses = []
losses.append(loss) # Keeps computation graph alive!

# Correct:
losses = []
losses.append(loss.item()) # Detach from graph

# Emergency cleanup:
import gc
torch.cuda.empty_cache()
gc.collect()
```

Hands-on Lab: Custom Training Loop

Lab Overview — 30 Minutes

This lab implements the same IMDB sentiment analysis task as Module 5, but using a fully manual PyTorch training loop. The goal is to understand what the Trainer abstraction does under the hood and develop the skills to extend or replace it when necessary.

1

Exercise 1

Basic custom training loop setup — data loading, model initialisation, optimiser and scheduler configuration

2

Exercise 2

Implement gradient accumulation to simulate a batch size of 32 on limited GPU memory

3

Exercise 3

Evaluation loop with accuracy, F1, and full classification report using sklearn

4

Exercise 4

Integrate mixed precision training with `autocast` and `GradScaler`

5

Exercise 5

Side-by-side comparison of custom loop vs. Trainer API on identical data and hyperparameters

Lab Exercise 1: Basic Custom Loop — Setup

```
print("=" * 60)
print("EXERCISE 1: Basic Custom Training Loop")
print("=" * 60)

import torch
from torch.utils.data import DataLoader
from transformers import (
    AutoModelForSequenceClassification,
    AutoTokenizer,
    get_scheduler,
)
from datasets import load_dataset
import numpy as np
from tqdm.auto import tqdm

# ——— 1. Device and model selection ———
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model_name = "distilbert-base-uncased" # Smaller model for faster lab execution
print(f"Using device: {device}")
print(f"Model: {model_name}")

# ——— 2. Load and tokenise a small subset ———
tokenizer = AutoTokenizer.from_pretrained(model_name)
dataset = load_dataset("imdb")
small_train = dataset["train"].select(range(1000))
small_test = dataset["test"].select(range(200))

def tokenize(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=128 # Short sequences for fast lab iteration
    )

train_tokenized = small_train.map(tokenize, batched=True, remove_columns=["text"])
test_tokenized = small_test.map(tokenize, batched=True, remove_columns=["text"])

train_tokenized.set_format("torch", columns=["input_ids", "attention_mask", "label"])
test_tokenized.set_format("torch", columns=["input_ids", "attention_mask", "label"])

# ——— 3. Create DataLoaders ———

train_loader = DataLoader(train_tokenized, batch_size=8, shuffle=True)
test_loader = DataLoader(test_tokenized, batch_size=16)
```

Lab Exercise 2: Training Loop with Gradient Accumulation

```
print("\n" + "=" * 60)
print("EXERCISE 2: Training with Gradient Accumulation")
print("=" * 60)

# ——— 3. Load model ———

model = AutoModelForSequenceClassification.from_pretrained(
    model_name, num_labels=2
)
model.to(device)

# ——— 4. Optimiser and scheduler ———

optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)
num_epochs = 3
accumulation_steps = 4
total_steps = (len(train_loader) // accumulation_steps) * num_epochs
warmup_steps = total_steps // 10

scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=warmup_steps,
    num_training_steps=total_steps,
)

# ——— 5. Training loop with gradient accumulation ———

model.train()
global_step = 0

for epoch in range(num_epochs):
    print(f"\nEpoch {epoch + 1}/{num_epochs}")
    epoch_loss = 0
    progress_bar = tqdm(train_loader, desc="Training")

    for step, batch in enumerate(progress_bar):
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        loss = outputs.loss / accumulation_steps
        loss.backward()
        epoch_loss += loss.item()

    if (step + 1) % accumulation_steps == 0 or (step + 1) == len(train_loader):
        optimizer.step()
        scheduler.step()
        optimizer.zero_grad()
        global_step += 1

    progress_bar.set_postfix({
        'loss': loss.item() * accumulation_steps,
        'lr': scheduler.get_last_lr()[0]
    })

avg_loss = epoch_loss / len(train_loader)
print(f"Epoch {epoch + 1} average loss: {avg_loss:.4f}")
```

Lab Exercise 3: Evaluation Loop

```

print("\n" + "=" * 60)
print("EXERCISE 3: Evaluation and Metrics")
print("=" * 60)

from sklearn.metrics import accuracy_score, f1_score, classification_report

def evaluate_model(model, dataloader, device):
    """Evaluate model and return loss, predictions, and ground-truth labels."""
    model.eval()
    all_predictions = []
    all_labels = []
    total_loss = 0

    with torch.no_grad():
        for batch in tqdm(dataloader, desc="Evaluating"):
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            total_loss += outputs.loss.item()
            predictions = torch.argmax(outputs.logits, dim=-1)
            all_predictions.extend(predictions.cpu().numpy())
            all_labels.extend(batch['labels'].cpu().numpy())

    avg_loss = total_loss / len(dataloader)
    accuracy = accuracy_score(all_labels, all_predictions)
    f1 = f1_score(all_labels, all_predictions, average='weighted')
    return avg_loss, accuracy, f1, all_predictions, all_labels

# ——— Run evaluation on test set ———
test_loss, test_acc, test_f1, predictions, labels = evaluate_model(
    model, test_loader, device
)

print(f"\nTest Results:")
print(f" Loss: {test_loss:.4f}")
print(f" Accuracy: {test_acc:.4f}")
print(f" F1 Score: {test_f1:.4f}")
print(f"\nClassification Report:")
print(classification_report(
    labels, predictions,
    target_names=['NEGATIVE', 'POSITIVE']
))

```

Lab Exercise 4: Adding Mixed Precision

```

print("\n" + "=" * 60)
print("EXERCISE 4: Adding Mixed Precision")
print("=" * 60)

from torch.cuda.amp import autocast, GradScaler

# ——— Re-initialise model and optimiser ———
model = AutoModelForSequenceClassification.from_pretrained(
    model_name, num_labels=2
).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)
scaler = GradScaler() # Manages dynamic loss scaling for FP16 stability

# ——— Training with mixed precision ———
model.train()
print("\nTraining with mixed precision...")

for epoch in range(2): # 2 epochs for demonstration
    progress_bar = tqdm(train_loader, desc=f"Epoch {epoch + 1}")
    epoch_loss = 0

    for step, batch in enumerate(progress_bar):
        batch = {k: v.to(device) for k, v in batch.items()}

        # FP16 forward pass
        with autocast():
            outputs = model(**batch)
        loss = outputs.loss / accumulation_steps

        # Scaled backward pass
        scaler.scale(loss).backward()
        epoch_loss += loss.item()

    if (step + 1) % accumulation_steps == 0:
        scaler.unscale_(optimizer) # Unscale before clipping
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(optimizer) # Skip if Inf/NaN in grads
        scaler.update() # Adjust scale factor
        scheduler.step()
        optimizer.zero_grad()

    progress_bar.set_postfix({'loss': loss.item() * accumulation_steps})

    print(f"Epoch {epoch + 1} loss: {epoch_loss / len(train_loader):.4f}")

# ——— Evaluate mixed-precision model ———
test_loss_fp16, test_acc_fp16, test_f1_fp16, _, _ = evaluate_model(
    model, test_loader, device
)
print(f"\nMixed Precision Results:")
print(f" Accuracy: {test_acc_fp16:.4f}")
print(f" F1 Score: {test_f1_fp16:.4f}")

```

Lab Exercise 5: Custom Loop vs. Trainer Comparison

```

print("\n" + "=" * 60)
print("EXERCISE 5: Custom Loop vs Trainer Comparison")
print("=" * 60)

from transformers import Trainer, TrainingArguments

# ——— Quick Trainer run for comparison ———
model_trainer = AutoModelForSequenceClassification.from_pretrained(
    model_name, num_labels=2
).to(device)

training_args = TrainingArguments(
    output_dir="./comparison",
    num_train_epochs=3,
    per_device_train_batch_size=8,
    gradient_accumulation_steps=4,
    learning_rate=2e-5,
    logging_steps=50,
    evaluation_strategy="no",
    report_to="none",
)

trainer = Trainer(
    model=model_trainer,
    args=training_args,
    train_dataset=train_tokenized,
)
trainer.train()

# ——— Evaluate Trainer model with the same evaluation function ———
_, trainer_acc, trainer_f1, _, _ = evaluate_model(model_trainer, test_loader, device)

# ——— Print comparison table ———

print(f"\nComparison Results:")
print(f"{'Metric':<15} {'Custom Loop':<15} {'Trainer API':<15}")
print(f"{'-' * 45}")
print(f"{'Accuracy':<15} {test_acc:<15.4f} {trainer_acc:<15.4f}")
print(f"{'F1 Score':<15} {test_f1:<15.4f} {trainer_f1:<15.4f}")

print(f"\nKey Takeaway:")
print(f" Both approaches achieve nearly identical performance.")
print(f" Trainer API is simpler but less flexible.")
print(f" Custom loops require more code but give full control.")

```

- ❏ The performance gap between both approaches should be negligible when hyperparameters are held constant. Any observed difference is attributable to stochastic variation in weight initialisation and data shuffling rather than a fundamental algorithmic difference.

Key Takeaways — Module 6

Custom Training Loops

Provide complete control over every step of the training process — essential for complex losses, custom gradients, and novel training regimes not supported by Trainer.

Essential Components

Every loop requires: forward pass, loss computation, `loss.backward()`, optional gradient clipping, `optimizer.step()`, and `scheduler.step()`.

Gradient Accumulation

Simulates larger batch sizes by accumulating gradients over multiple forward passes. Always divide the loss by the accumulation steps to preserve gradient magnitude.

Mixed Precision (FP16)

Delivers 2–3× training speedup and 50% memory reduction with negligible accuracy impact. Requires `GradScaler` to prevent gradient underflow.

Debugging

Gradient clipping (`max_grad_norm`) cures NaN losses; anomaly detection pinpoints the operation; detaching tensors with `.item()` prevents memory leaks.

Trainer vs. Custom

Choose Trainer for standard tasks and rapid iteration. Choose custom loops when you need fine-grained control, complex losses, or integration with non-HuggingFace code.

Performance Tips

01

Gradient Accumulation

Always use when GPU memory limits your target batch size

02

Mixed Precision

Enable FP16 on all Volta+ GPUs for free speedup

03

Pre-tokenise

Save tokenised datasets to disk and reload — avoid re-tokenising per run

04

Eval Batch Size

Use 2–4× larger batches during evaluation (no activations to store)

05

Gradient Norms

Monitor gradient norms every step to catch instability early

Additional Resources — Module 6

Official Documentation

- [**PyTorch Training Loop Guide**](#) — Step-by-step introduction to building training loops from scratch
- [**Mixed Precision Training \(torch.cuda.amp\)**](#) — Official AMP documentation with examples for autocast and GradScaler
- [**Gradient Accumulation Guide**](#) — HuggingFace Accelerate's deep dive into gradient accumulation semantics

Tutorials & Tools

- [**HuggingFace NLP Course — Chapter 3**](#) — Covers both Trainer-based and custom loop approaches
- [**Custom Training with Transformers**](#) — Practical guide to custom datasets and training scripts
- [**Accelerate Library**](#) — Single-line abstraction for multi-GPU, TPU, and distributed training
- [**PyTorch Profiler**](#) — Identify bottlenecks in your training loop with operator-level profiling

Day 3 Complete Recap

MODULE 5

The Trainer API & Full Fine-Tuning

Fine-tuning fundamentals and transfer learning principles

Data preparation, tokenisation strategies, and stratified splitting

Deep dive into `TrainingArguments` — every key parameter explained

Trainer API with metrics, callbacks, TensorBoard, and checkpointing

Common issues — OOM, overfitting, slow training, instability — and their solutions

Hands-on: Full BERT fine-tuning pipeline on IMDB — data → train → evaluate → save

MODULE 6

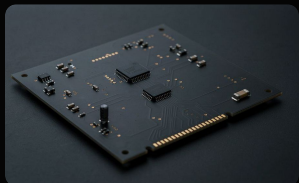
Training Without the Trainer

- When custom loops are necessary vs. when Trainer suffices
- Anatomy of a PyTorch training loop — all five core components
- Reusable `train_epoch` and `evaluate` function templates
- Gradient accumulation — theory, correct loss normalisation, and implementation
- Mixed precision training with `autocast` and `GradScaler`
- Hands-on: Custom loop with all optimisations — side-by-side comparison with Trainer

Tomorrow — Day 4 Preview

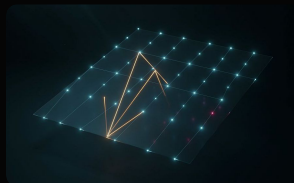
Parameter-Efficient Fine-Tuning and Quantisation

Day 4 builds directly on today's fine-tuning foundations and introduces techniques that make fine-tuning accessible on consumer hardware — including a single 24GB or even 16GB GPU.



Parameter-Efficient Fine-Tuning (PEFT)

Freeze the pre-trained backbone and train only a tiny set of additional parameters — adapters, prefix tokens, or low-rank matrices. Achieves competitive performance with 99%+ fewer trainable parameters.



LoRA: Low-Rank Adaptation

Decompose weight update matrices into two low-rank factors. Trains only the rank-decomposition matrices while the original weights remain frozen. State-of-the-art efficiency for LLM fine-tuning.



Quantisation: 4-bit and 8-bit

Represent model weights in lower-bit integer formats (INT8, INT4) to dramatically reduce memory footprint and inference latency with minimal quality degradation using calibration techniques.



QLoRA: Efficient Fine-Tuning on Consumer GPUs

Combines 4-bit quantisation with LoRA to enable fine-tuning of 7B–70B parameter models on a single consumer GPU. Democratises LLM customisation for individual researchers and small teams.